# An Extension of Buddy Systems to Arbitrary Radix for Deterministic Memory Allocation

**Andrew Jonathan Fine**

May 31, 2026

**Abstract**

Traditional heap management does not terminate predictably: the cost of its search depends on the state of the heap being searched. This paper introduces Iscandar, a deterministic memory allocator built as a radix-32 partitioned lattice. A request is resolved not by searching the heap but by examining a fixed grid of occupancy counters in a fixed order, consulting precomputed oracle tables for the smallest request classes, and committing at a single point. The work of every request class is bounded by a ceiling fixed by the radix alone, and those ceilings are theorems about the construction, not measurements. With the two-pass construction completed in this paper, a placement exists if and only if a contiguous free run of the request exists. Allocations are exact: no sector is rounded away to a size class. Measured against a buddy allocator on identical request streams, the design trades extra examinations for certainty: costs and placements that reproduce exactly, and a largest usable hole that survives far longer. Cheapness per operation is not offered, and none is claimed.

# Contents


**1 Welcome to Iscandar. 1**

**2 Architectural Axioms 1**
- 2.1 Axiom I: The State Map (L) and Structural Arity ($\mathcal{A}$) . . . 1
  - 2.1.1 The Cube Namespace . . . 1
- 2.2 Axiom II: Hierarchical Occupancy Counters ($\Gamma$) . . . 1
- 2.3 Axiom III: The Contiguity Guarantee and Spatial Entropy . . . 1
- 2.4 Axiom IV: The Coordinate-Address Identity . . . 2

**3 Formal Proof of Deterministic Path Termination 2**
- 3.1 The Steering-Based Complexity Model . . . 2
- 3.2 Guaranteed Path Termination . . . 3
- 3.3 Conclusion of Determinism . . . 3

**4 Deterministic Coalescence 3**
- 4.1 Symmetric Coalescence Mechanics . . . 3
- 4.2 Proof of Coalescence Invariant . . . 4

**5 Observations 4**
- 5.1 As an Extension to Binary Buddy Systems . . . 4
- 5.2 As a Symmetric Voronoi Partition . . . 4
- 5.3 As Pre-Qualification by Counters . . . 4
- 5.4 Scalability via Depth Extension . . . 5
- 5.5 As Supporting Parallel Execution . . . 5

**6 Conclusion 5**

**7 Glossary of Terms 6**

**8 Updating Memory State Spaces by Mating the Oracle Tables 6**
- 8.1 Polytope Formation at Cube Level . . . 6
  - 8.1.1 The Cube Scale Allocator ($L = \mathcal{A}^k$) . . . 6
- 8.2 Polytope Formation At Row and Plane Levels . . . 7
  - 8.2.1 The Huge Resolution ($\mathcal{A}^2 < L < \mathcal{A}^3$) . . . 7
  - 8.2.2 The Large Resolution ($2\mathcal{A} < L \leq \mathcal{A}^2$) . . . 8
- 8.3 Polytope Formation Between Rows . . . 10
  - 8.3.1 The Small Resolution ($A < L \leq 2\mathcal{A}$) . . . 10
- 8.4 Polytope Formation Within Row . . . 11
  - 8.4.1 The Tiny Resolution ($\mathcal{A}/4 < L \leq \mathcal{A}$) . . . 11
  - 8.4.2 The Nano Resolution ($L \leq \mathcal{A}/4$) . . . 11

**9 Bounded Complexity by Bounding Execution Paths 12**
- 9.1 Mechanics of Guaranteed Termination . . . 12

**10 Desk Check Summary: Nano and Tiny Resolutions ($L \leq \mathcal{A}$) 12**
- 10.1 The Nano Trace ($L = 5$) . . . 13
- 10.2 The Tiny Trace ($L = 11$) . . . 13
- 10.3 Verification . . . 13

**11 Volumetric Analysis 14**
- 11.1 Methodology . . . 14
- 11.2 Graphical Verification . . . 14

*To Jennifer:*

*Everything. Always. Forever.*

# 1 Welcome to Iscandar.

Unlike allocation models using variable-time search algorithms [1], this architecture[1] employs a hierarchical occupancy map to resolve coordinates deterministically. Extending the traditional binary buddy system to an arbitrary radix replaces it with a geometric steering mechanism.

# 2 Architectural Axioms

These axioms establish the geometric and mathematical constraints of the lattice. They transform each memory request into a coordinate resolution task, avoiding search-and-sort operations to maintain a fixed sequence of instructions.

## 2.1 Axiom I: The State Map (L) and Structural Arity ($\mathcal{A}$)

For modeling and testing purposes this paper demonstrates an $k$-level lattice $L$ of fixed depth ($n = 3$) defining a total memory space. The architecture maintains a constant Arity ($\mathcal{A}$) of 32. A hierarchical progression of tiers organizes the lattice: the atomic Sector ($S_0$), the Row of Sectors ($S_1$), the Plane of Rows ($S_2$), and the Cube of Planes ($S_3$). Lattice coordinates $(p, r, s)$ determine the address of any specific leaf node, where $p, r, s \in \{0, \ldots, \mathcal{A} - 1\}$. The geometry maintains invariance regardless of the physical byte-width the sector occupies.[2]

### 2.1.1 The Cube Namespace

The Cube defines the memory allocator data structure, Iscandar's answer to the buddy system heap, in accordance with Axiom I.

## 2.2 Axiom II: Hierarchical Occupancy Counters ($\Gamma$)

Discrete occupancy counts ($\Gamma$) at every tier enable constant-time steering. Following bit-block representation principles, these counters generate a real-time density map of the lattice:

$$\text{Level 1 (Row):} \quad \Gamma_1 = \sum_{i=0}^{\mathcal{A}-1} \Gamma_{0,i} \tag{1}$$

$$\text{Level 2 (Plane):} \quad \Gamma_2 = \sum_{j=0}^{\mathcal{A}-1} \Gamma_{1,j} \tag{2}$$

$$\text{Level 3 (Cube):} \quad \Gamma_3 = \sum_{k=0}^{\mathcal{A}-1} \Gamma_{2,k} \tag{3}$$

This hierarchical summation lets the Cube level perform its admissibility check as a single comparison against the apex counter, imposing a fast-fail condition.

## 2.3 Axiom III: The Contiguity Guarantee and Spatial Entropy

Any resolved coordinate sequence represents a continuous physical block in the **Iscandar** lattice. However, functional contiguity does not imply spatial perfection; a perfect Oracle would preclude fragmentation entirely. Our Oracles (Center, Left, and Right) prioritize determinism over maximum packing density.

[1] The name is a nod to Star Blazers. Iscandar is the distant world whose gift descends, complete and ready, to a world that cannot save itself. The design principle taken from it: the allocator is meant to drop in, install at once, and work with a minimum of instruction, as a Wave Motion Core does.

[2] The Large allocator resolves sequences across rows, and across at most one plane boundary, to satisfy multi-Row requests.

### 2.4 Axiom IV: The Coordinate-Address Identity

The architecture used here rejects the traditional separation of logical address space and physical memory geometry, by establishing a **Bijective Identity** between a machine-word pointer $P$ (wrapped pointer, see Glossary) and its corresponding lattice coordinate $\vec{C}$.

In a system of depth $k$ and arity $A$, any specific sector is represented by a coordinate vector:

$$\vec{C} = (i_1, i_2, \ldots, i_k) \quad \text{where} \quad 0 \leq i_n < A \tag{4}$$

The address $P$ is not an arbitrary token, but the scalar projection of this vector into the linear address space. The isomorphism is defined by the construction:

$$P = \sum_{n=1}^{k} i_n \cdot A^{k-n} \tag{5}$$

With our cube as an example, the address of a sector located at $(p, r, s)$ is clearly enumerable from the geometry:

$$P = (p \cdot 32^2) + (r \cdot 32^1) + (s \cdot 32^0) \tag{6}$$

The construction is a bijection between the coordinate vectors and the addresses $0 \leq P < \mathcal{A}^k$. Distinct coordinates give distinct addresses: if two vectors first differ at position $n$, their addresses differ by a nonzero multiple of $\mathcal{A}^{k-n}$, while all later positions together contribute less than $\mathcal{A}^{k-n}$ in absolute value ($\sum_{m>n}(\mathcal{A} - 1)\,\mathcal{A}^{k-m} = \mathcal{A}^{k-n} - 1$), so the difference cannot cancel; this is the uniqueness of the base-$\mathcal{A}$ expansion. There are $\mathcal{A}^k$ coordinate vectors and $\mathcal{A}^k$ addresses in the range, and an injective map between finite sets of equal size is onto, so every address arises. The inverse is repeated division: $i_k = P \bmod \mathcal{A}$, then divide by $\mathcal{A}$ and repeat, recovering $i_{k-1}, \ldots, i_1$ in turn.

## 3 Formal Proof of Deterministic Path Termination

**Theorem:** A fixed-depth lattice $L$ of Arity $\mathcal{A} = 32$ establishes deterministic resolution: every memory request is assigned by decision table to exactly one resolution path; that path examines suspects in a fixed order, halts at the first conviction or at exhaustion, and its number of trials is bounded by a ceiling fixed by $\mathcal{A}$ alone. Identical requests against identical occupancy states execute identical steps and return identical results.

### 3.1 The Steering-Based Complexity Model

The resolution time $T$ in the Iscandar architecture is a discrete function of the sub-lattice depth and the specific heuristic path $P$ identified by the memory request size in sectors. Unlike legacy systems where $T = \mathcal{O}(\log N)$ or $\mathcal{O}(N)$ based on heap fragmentation, the path selection in Iscandar uses a decision table to ensure every heuristic terminates in bounded time.

Let $\mathcal{H}$ be the finite set of available resolution paths. The path is selected by decision table on the request length $L$ alone; the occupancy state $\Gamma$ does not choose the path. It governs admissibility, and which suspects the chosen path examines:

$$P(L) \in \mathcal{H} = \{P_{\text{Nano}}, P_{\text{Tiny}}, P_{\text{Small}}, P_{\text{Large}}, P_{\text{Huge}}, P_{\text{Cube}}\} \tag{7}$$

A **trial** is one examination of one suspect: a counter qualification, with at most one Oracle consult per suspect for Nano and two for Tiny. A path whose lanes nest $d$ levels deep performs at most $\mathcal{A}^d$ trials before exhaustion, and halts earlier at the first conviction. Writing $\tau$ for the fixed cost of one trial, the execution time $T$ (measured in operational steps) obeys

$$T(P) \;\leq\; \tau \cdot \text{trials}(P) \;\leq\; \tau \cdot \mathcal{A}^d, \tag{8}$$

with worst-case trial ceilings by path:

| Path | Trial ceiling |
|---|---|
| Cube | 1 admissibility comparison |
| Nano | $\mathcal{A}^2 = 1{,}024$ row trials |
| Tiny | $\mathcal{A}^2 = 1{,}024$ row trials ($\leq 2$ consults each) |
| Small | $\mathcal{A}^2 = 1{,}024$ row-pair trials |
| Large | $2\mathcal{A}^2 = 2{,}048$ row trials (two passes) |
| Huge | $2\mathcal{A}^2 = 2{,}048$ row trials (two sweeps) |

### 3.2 Guaranteed Path Termination

Every resolution path is constructed from a fixed sequence of instruction groups that mathematically eliminate runtime searching.

1. **Heuristic Selection:** The decision table assigns the path $P \in \mathcal{H}$ by request length alone. The Global Registrar then applies the top-level admissibility check against the apex counter before any descent.
2. **Atomic Resolution:** For leaf-level requests, the system executes either a single-pass extraction (Nano), a dual-pass Left-then-Right oracle extraction (Tiny), or a tripartite join across a row pair (Small).
3. **Fixed Order, Early Halt:** Once a path is engaged, it examines its suspects in a fixed lane order and halts at the first conviction, committing exactly once. If no suspect convicts, the path returns its defined failure at exhaustion.

### 3.3 Conclusion of Determinism

Determinism in Iscandar is the conjunction of two guarantees. The first is **reproducibility**: an identical request against an identical occupancy state follows the identical path, executes the identical steps, and returns the identical result. The second is **boundedness**: every path terminates within its class's trial ceiling, at a conviction or at exhaustion, with no open-ended search. The executed step count varies with the occupancy state; the ceiling does not. By relying on unrolled geometric coordinate resolution, the architecture deliberately trades away maximum spatial packing density in exchange for this bounded, reproducible termination.

## 4 Deterministic Coalescence

Maintaining temporal determinism in a dynamic environment requires a deallocation process (coalescence) mirroring the efficiency of the allocation logic.

### 4.1 Symmetric Coalescence Mechanics

To maintain determinism, our deallocation process mirrors the efficiency of the allocation logic via a direct state update performed upon the occupancy counters. The wrapped pointer explicitly carries the allocation's origin and length (see Axiom IV), so deallocation needs no search: it sweeps exactly the sectors the pointer names. All counter updates, for allocation and deallocation alike, are performed at a single commitment point, which is what keeps the counter hierarchy consistent. Updating the occupancy counters ($\Gamma$) is thus the dual to the initial resolution:

1. The deallocation interface (`UnMark.api`) is invoked directly via the wrapped pointer. Each sector of the extent has its leaf node state $\Gamma_0$ set to 0 (Available).
2. Each affected Row counter ($\Gamma_1$) is decreased by the number of sectors freed in that row.
3. Each affected Plane counter ($\Gamma_2$) is decreased by the number of sectors freed in that plane, and the Cube counter ($\Gamma_3$) by the length of the extent.

The sweep performs one leaf update per freed sector and one counter update per affected row and plane. Its cost is therefore fixed by the extent length and the depth $n = 3$, and never by the occupancy state of the rest of the cube.

### 4.2 Proof of Coalescence Invariant

Let $\Gamma_{\text{Apex}}$ denote the cube counter $\Gamma_3$, the total occupancy of the cube. Each leaf value $\Gamma_0$ is a single occupancy bit, 0 or 1. At any discrete time $t$, the hierarchical counters satisfy the invariant:

$$\Gamma_{\text{Apex}} = \sum \Gamma_2 = \sum \Gamma_1 = \sum \Gamma_0 \tag{9}$$

Deallocation sets each freed leaf bit to 0 and decreases every counter by the number of freed sectors inside the unit it counts:

$$\Gamma_x(t+1) = \Gamma_x(t) - f_x \quad \text{for counter levels } x \in \{1, 2, 3\} \tag{10}$$

where $f_x$ is the number of freed sectors falling inside the unit that $\Gamma_x$ counts: the freed sectors of one row for $x = 1$, of one plane for $x = 2$, and the full extent length for $x = 3$. The freed counts sum identically at every level: the row counts add to the plane counts, and the plane counts to the extent length. Every term of the invariant therefore decreases by the same total, and the equality is preserved. Allocation is the same update with the sign reversed. Because updates occur only at the single commitment point, no intermediate state is ever observed in which the levels disagree.

## 5 Observations

Earlier sections established **Iscandar** as deterministic within the primary scope of this paper. However, it exhibits properties warranting distinct architectural observations.

### 5.1 As an Extension to Binary Buddy Systems

The binary buddy system, pioneered by Knowlton [12] and Knuth [13], sought to minimize search latency through a power-of-two partitioning scheme ($R = 2$). While this approach facilitates efficient coalescence, its cost depends on the state of its free lists. At high utilization the search must pick through increasingly fragmented blocks, and the time it takes varies with the heap's history: the "stalling" behavior that erodes deterministic performance.

We generalize buddy system principles to an **Arbitrary Radix** ($R = \mathcal{A}$). By shifting from a binary tree to a high-radix fractal lattice, we collapse the depth of the search space to a fixed architectural constant $n$. This structural shift replaces the search through free blocks with the resolution of a coordinate against the occupancy counters.

### 5.2 As a Symmetric Voronoi Partition

In this model, adjacency is defined by a partition's total volume associating with its handle (the wrapped pointer of Axiom IV) rather than by the Euclidean distance of individual points from a centroid.

In a standard Voronoi partition, boundaries are dynamically computed as a function of the Euclidean distance to a specific "seed" or generator point. In the Iscandar lattice, the "seed" is represented by the wrapped pointer, but the partition boundaries are not computed at runtime. They are architecturally invariant, defined strictly by the Structural Arity ($\mathcal{A}$) and the hierarchical depth ($k$) established in the Axioms. Admission of a request is therefore a function of the request length and the occupancy counters alone: no distance between any position and any previous allocation enters the decision.

### 5.3 As Pre-Qualification by Counters

The occupancy counters pre-qualify every region before the descent enters it: a plane or row whose counter cannot support the request is never examined further, so the resolution path never enters a region that cannot satisfy the request. Fragmentation is met at the counter level, by steering, and at the row level, by oracle trial. The path is kept out of the fragmentation trap by construction, not by navigating around it.

### 5.4 Scalability via Depth Extension

The **Iscandar** architecture scales by increasing the lattice depth up to two additional levels. Because the resolution time is a linear function of depth ($k \times I_{base}$), extending the lattice to $k = 4$ or $k = 5$ multiplies the addressable memory volume by $\mathcal{A}$ with each added level, while maintaining a strictly deterministic and predictable execution profile.

### 5.5 As Supporting Parallel Execution

The architecture allows for the distribution of the management load through parallel instantiation. Lesser structures can be managed by hardware subroutines to distribute the processing requirements across multiple units. These parallel hardware components can also supervise local memory independently, communicating management data upward through the hierarchy only as required.

## 6 Conclusion

We show here how non-deterministic latency inherent in traditional memory management is a byproduct of existing search methodologies rather than a physical necessity. Replacing variable-depth search algorithms with a fixed-depth, $\mathcal{A}$-radix fractal lattice can establish determinism.

What the design sells is certainty, and the implementation's measurements support it in four respects.

First, exact fit. An Iscandar allocation occupies exactly the sectors requested: no rounding to a size class, no internal waste. In the Gaussian Two hold, the buddy allocator rode at 73% physical occupancy to carry a 50% workload, some 23 points of the pool burned as rounding, peaking near a third; Iscandar's physical line lay on the workload line for 20,000 operations. In a 128 KiB pool, that waste is the difference between a system that fits and one that does not. The buddy allocator exhausted at 100% physical in two different load ramps while Iscandar, on identical request streams, sat at 63 to 76%.

Second, reproducibility. Three of the six measured load profiles returned variance exactly zero over as many as 100,000 operations: the same cost, the same placement, and the same stall point for the same request stream, every time. That is the property flight code is starved for: ground testing that reproduces flight behaviour exactly.

Third, provable ceilings. The cost of a search is bounded by the trial ceilings of Section 3, which are theorems about the construction, not statistics about samples: at a request of 1,025 sectors the bound of 70 touches is a theorem: no occupancy state can exceed it. A real-time engineer budgets worst cases, not averages. A conventional allocator hands out blocks and certifies nothing; Iscandar pays its extra touches to verify what it gives.

Fourth, promises kept longer. At half occupancy, Iscandar's largest usable hole ran about three times the buddy allocator's, and held long: capacity promised early remains available late, because nothing was rounded away in the meantime.

What the design does not sell is raw cheapness per operation, and no universality is claimed for its consistency: under a bimodal mixed load, cross-mode interference appears and the buddy allocator won that sprint outright. The consistency lives in the variance being zero where the design says it should be, and in bounds that are proven rather than sampled. In the author's summary: the allocator is angelic for determinism, and everything else is a bonus.

The allocator description is published in the software project on GitHub; this monograph presents the theory of it.

# 7 Glossary of Terms

**Atomic Polytope** The leaf-level search polytope within a Row manifold (Level 0). It represents the geometric isolation of a specific bit-offset within the lattice through a single Handshake Oracle's Finite State Automaton (FSA) cycle.

**Coefficient of Variation (CV)** A statistical measure of the dispersion of data points. In the **Iscandar** context, a CV of zero in a measured profile means the measured cost did not vary across that profile's operations; it is a property of the measurement, not of every execution. [15]

**Isochronous** Executing within a work bound that depends only on the request class and the radix, never on the heap's history. The word is used in this paper in that bounded sense; identical timing on every call is neither claimed nor required.

**Contiguity Guarantee** The formal assurance (Section 2.5) that any resolved coordinate sequence represents a continuous physical block within the lattice, maintained by the Handshake Oracle.

**Global Registrar** The Level 3 (Cube Scale) apex of the hierarchy. It holds the apex counter $\Gamma_3$, the cube's total occupancy as an unsigned value, and performs the admissibility check before the algorithm permits a hierarchical descent.

**Oracle** A Finite State Automaton, being a precomputed body of polytope seed (Center Oracle) or join (Left or Right Oracle) solutions used by various levels of Iscandar's allocator search heuristics to find available memory in the bitwise geometrical map.

**Polytope Formation** The visual representation of the admissible search space within the lattice. Each hierarchical tier (Cube, Plane, Row) can manifest polytopes that collapse as memory occupancy increases.

**Occupancy Limit ($\Phi$)** The mandatory occupancy boundary (Section 4.2) that prevents the system from entering a high-entropy state. Once the floor is reached, the system issues a deterministic refusal rather than engaging in variable-time search.

**Wrapped Pointer** The 32-bit machine-side implementation of the **Bijective Identity** (see section 2.4, Axiom IV). It bridges between the memory heap coordinate and a physical MPU.

| Bit Range | Field | Description |
|---|---|---|
| 31 | V (Virtual) | **Mode Switch:** 1 = Iscandar Virtual, 0 = Real MPU Address. |
| 30 | R (Reserved) | Reserved for future architectural parity. |
| 15–29 | S (Start) | 15-bit Coordinate Index (Plane, Row, Sector). |
| 0–14 | L (Length) | 15-bit Allocation length/depth. |

# 8 Updating Memory State Spaces by Mating the Oracle Tables

This section describes how **Iscandar** resolves memory requests by intersecting partitioned sub-spaces. The system utilizes primary ratios relative to the structural Arity ($\mathcal{A}$): the Guard Shards ($0.5\mathcal{A}$) and the Core Shard ($0.25\mathcal{A}$).

## 8.1 Polytope Formation at Cube Level

### 8.1.1 The Cube Scale Allocator ($L = \mathcal{A}^k$)

**Domain:** Total Cube Volume resolution (addressable at $\mathcal{A}^k$).

**Discussion:** The Cube Scale Allocator computes the unique **"Gatekeeper"** sector count of the architecture. It provides the primary fast-fail heuristic for the system; if a request $(S, M)$ is determined to be non-admissible at this tier, the algorithm immediately terminates without initiating a descent.

## 8.2 Polytope Formation At Row and Plane Levels

### 8.2.1 The Huge Resolution ($\mathcal{A}^2 < L < \mathcal{A}^3$)

**Domain:** Multi-planar sequence resolution across the global $\mathcal{A}^k$ Cube row-manifold.

**Decomposition:** Write core $= \lfloor L/\mathcal{A}^2 \rfloor$ for the whole planes of the request and $R = L \bmod \mathcal{A}^2$ for the remainder. Every contiguous span of length $L$ decomposes as a take from a head plane, $f$ whole empty planes, and a take from a trailer plane:

$$L = h + f\mathcal{A}^2 + t, \qquad f \in \{\text{core} - 1,\ \text{core}\} \tag{11}$$

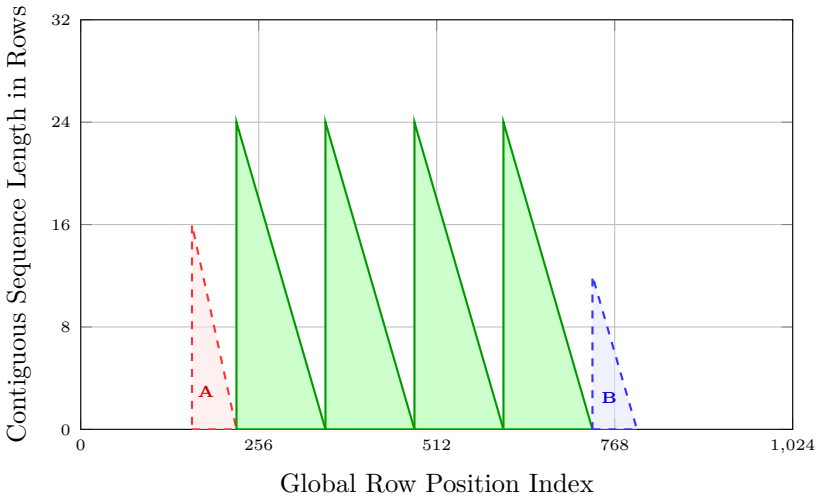
with $h + t = R$ when $f = \text{core}$, and $h + t = R + \mathcal{A}^2$ when $f = \text{core} - 1$. The middle planes are whole exactly when their counters read 0; the two takes are certified by the same row machinery as the Large tier, one level down.

**Discussion:**

- **Mechanism:** Unrolled evaluation of cube's empty rows with Oracle based partial row evaluation.
- **Geometric Event:** Allocation from sequence of partial and empty rows.
- **Resolution:** The request is decomposed across global row indices, in the form of the Decomposition above. A first sweep searches for the $f = \text{core}$ form: a sequence of wholly empty planes, verified by their counters, flanked by the head and trailer takes. If the first sweep finds nothing, a second sweep searches the split seams for the $f = \text{core} - 1$ form.

The Huge allocation exceeds the capacity of the standard Large allocator. It evaluates the global row coordinate landscape linearly from 0 to 1,024, so a span may cross a plane boundary as readily as a row boundary, with the seams handled by the two sweeps of the Decomposition above.

The second sweep exists because the first sweep cannot see the split form. Its anchors are the seams: core $- 1$ consecutive empty planes flanked by a head plane and a trailer plane. The head take is computed by the normal handler's machinery with its budget raised to $R + \mathcal{A}^2$ (the remainder plus one whole plane), and the trailer's fulfilment of what remains is mandatory. A split acquittal returns the dedicated status `HUGE_SPLIT_FAIL` $= 19$. When core $= 1$ the run of whole planes is empty and the seam is a bare plane boundary: a suffix of one plane welded to a prefix of the next. Example: a request of 2,049 whose only free run stands as $1{,}023 + 1{,}024 + 2$ across three planes has core $= 2$ and $R = 1$; the first sweep finds no two whole planes inside the run, and the second sweep places it with $f = 1$ and edge takes summing to $R + \mathcal{A}^2 = 1{,}025$. The first sweep is unchanged by the amendment, so every placement it made before is made identically after it.




**(A)**

Figure 1: Example of Huge memory allocation: Series of central polytopes revealed by cumulative empty row counts joined by optional head (A) and tail (B) polytopes revealed by Left and Right Oracles.

### 8.2.2 The Large Resolution ($2\mathcal{A} < L \leq \mathcal{A}^2$)

**Domain:** Row-join and sequence resolution within an $\mathcal{A}^2$-sector Plane manifold.

**Decomposition:** Write core $= \lfloor L/\mathcal{A} \rfloor$ for the whole rows of the request and $r = L \bmod \mathcal{A}$ for the remainder. Every contiguous span of length $L$ decomposes as a trailing take from a head row, $f$ whole empty rows, and a leading take from a tail row:

$$L = h + f\mathcal{A} + t, \qquad f \in \{\text{core} - 1,\ \text{core}\} \tag{12}$$

with $h + t = r$ when $f = \text{core}$, and $h + t = r + \mathcal{A}$ when $f = \text{core} - 1$. The whole rows are exactly the rows whose counters read 0; the takes are certified by the Left and Right Oracles.

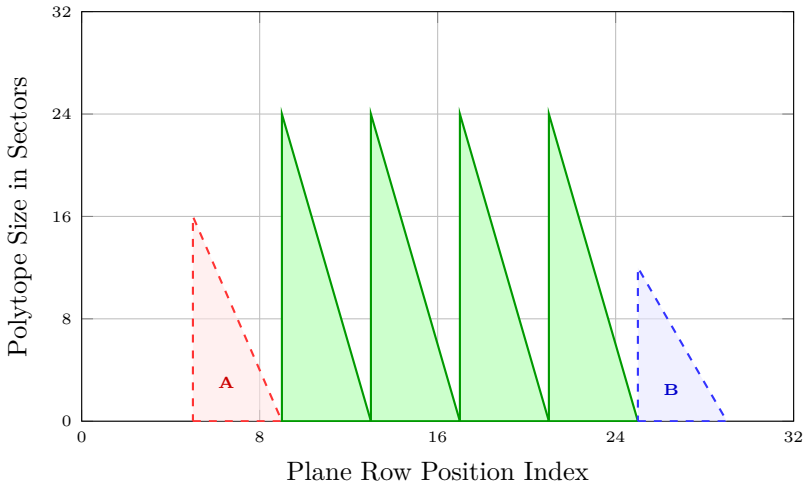
**Discussion:**

- **Mechanism:** The *Structural Bridge.*
- **Geometric Event:** *Coalescence between empty and partial rows.*
- **Resolution:** The system constructs the span via $P + T$. The Right-Hand Lookup (RHL) of Vector A (Preamble) mates with the Left-Hand Lookup (LHL) of Vector B (Tail) to verify connectivity across the boundary, a geometric invariant enforced by these tables.

This tier applies the same construction one level up: rows take the place of sectors as the units of resolution, and row sequences are welded to satisfy multi-row allocation requests. The Large search assembles the rows of a span within the plane, and across at most one plane boundary, under the decomposition above.

The search runs in two passes over the same launchings, in the same order. The first pass seeks the $f = \text{core}$ form, examining each candidate launching with a rolling window of core whole rows and settling the remainder against the edge rows, head first and then tail. It is unchanged from the original construction. The second pass runs only when the first finds nothing, and seeks the split form: a rolling window of core $- 1$ whole rows under a unified edge budget of $r + \mathcal{A}$, in which the head row's trailing run is taken whole toward the budget and the tail row's leading run must cover the rest. Its launch gate carries one allowance the first pass's gate does not: the free count of the single head row before the launching plane, because a split span can lean up to $\mathcal{A} - 1$ sectors into that row and the two-plane sum never counts it. Example: a request of 100 has core $= 3$ and $r = 4$; a free run standing as $31 + 32 + 32 + 5$ decomposes with $f = 2$ and edge takes summing to $36 = r + \mathcal{A}$, and is placed by the second pass after the first pass exhausts.

**Completeness.** With both passes in place at each tier, a placement exists if and only if a contiguous free run of the request exists. The decomposition fact is what closes the argument: a contiguous span's whole-unit count $f$ takes no value other than core or core $- 1$, the first pass covers the one and the second pass the other, at every alignment. Before the amendment, a span of the split form exhausted the search while an exact-fit hole stood open; the amendment removes that case at both the Large and the Huge tier.




**(B)**

Figure 2: Example of Large memory allocation: Series of central row-polytopes revealed by cumulative empty sector counts inside a single Plane, joined by optional head and tail fragments revealed by Left and Right Oracles.

## 8.3 Polytope Formation Between Rows

### 8.3.1 The Small Resolution ($A < L \leq 2\mathcal{A}$)

**Domain:** Sector-join and sequence resolution across a pair of adjacent Rows, $j$ and $j+1$.

**Summation Predicate:**

$$C_{row} = \sum_{j=0}^{\mathcal{A}-1} (C_{bits,j} \cdot [x_j = 1]) \tag{13}$$

**Join Predicate:** the tripartite join of the Resolution item is the equation

$$L \;=\; \mathrm{RHL}(W_j) \;+\; \mathrm{LHL}(W_{j+1}) \tag{14}$$

where $\mathrm{RHL}(W_j)$ is the trailing free run of the first row and $\mathrm{LHL}(W_{j+1})$ the leading free run of the next, each measured from the shared boundary, with the Center consult certifying that no occupied sector interrupts either run. The join is admissible exactly when the two runs sum to at least $L$; the allocation then begins at position $\mathcal{A} - \mathrm{RHL}(W_j)$ of the first row.

**Discussion:**

- **Mechanism:** *Tripartite Join*
- **Geometric Event:** *Polytope Intersection*
- **Resolution:** The Oracle pieces together the Right-Hand Lookahead (RHL) of the first Guard, the internal gaps of the Center Lookahead, and the Left-Hand Lookahead (LHL) of the second Guard, to verify a contiguous span of available sectors, representing a complete geometric resolution within the Oracle's combinatorial state-space.

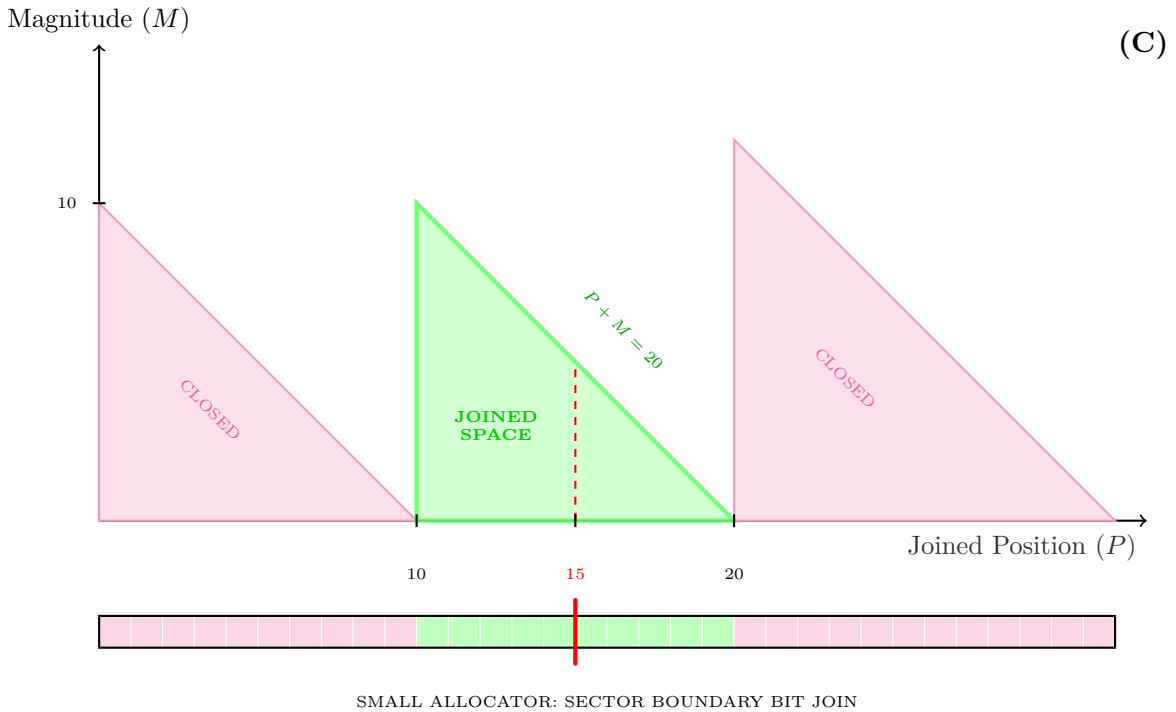


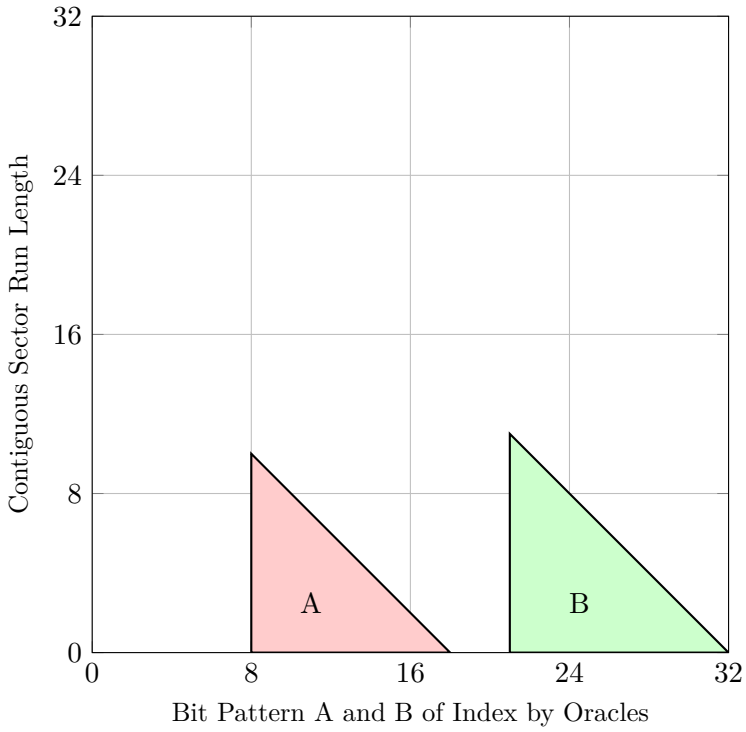

Figure 3: Small Allocator: Atomic Phase-Space and Sector Boundary Bit Join.

### 8.4 Polytope Formation Within Row

#### 8.4.1 The Tiny Resolution ($\mathcal{A}/4 < L \leq \mathcal{A}$)

**Discussion:**

- **Mechanism:** This targets a request in sectors exceeding the single-byte Nano threshold but still fitting within a Row's width.
- **Geometric Event:** A *Bimodal Interior* query.
- **Resolution:** The resolution logic first applies the **Left Oracle** to identify an eligible run of at least $\mathcal{A}/4$ sectors. This polytope can be revealed by this Oracle as anywhere in the row. Should that fail, the system takes a "second bite of the apple" by immediately applying the **Right Oracle**. This polytope when present is revealed as always on the right hand side of the row. This enhances captures of straddles within the row.




**(D)**

Figure 4: *
Tiny mode seed polytopes A and B from indexing Left and Right Oracles.

#### 8.4.2 The Nano Resolution ($L \leq \mathcal{A}/4$)

**Domain:** The bit-lattice contained within a single word or sub-word segment of a Row manifold, specifically targeting requests between 1 and 8 bits ($\mathcal{A}/4$ when $\mathcal{A} = 32$).

**Discussion:**

- **Mechanism:** The Center Oracle is consulted on the Row's full 32-bit occupancy word. It returns the leftmost position at which $L$ consecutive sectors are free, or failure if no such run exists.
- **Geometric Event:** An *Intersection of Atomic (Seed) Polytopes*.
- **Resolution:** The **Center Oracle** intersects the current Row's bit-pattern with the requested length $L$ and returns an exact placement of 1 to 8 sectors within the row, or acquits the row. A conviction ends the search; an acquittal resumes the procession at the next qualifying row.

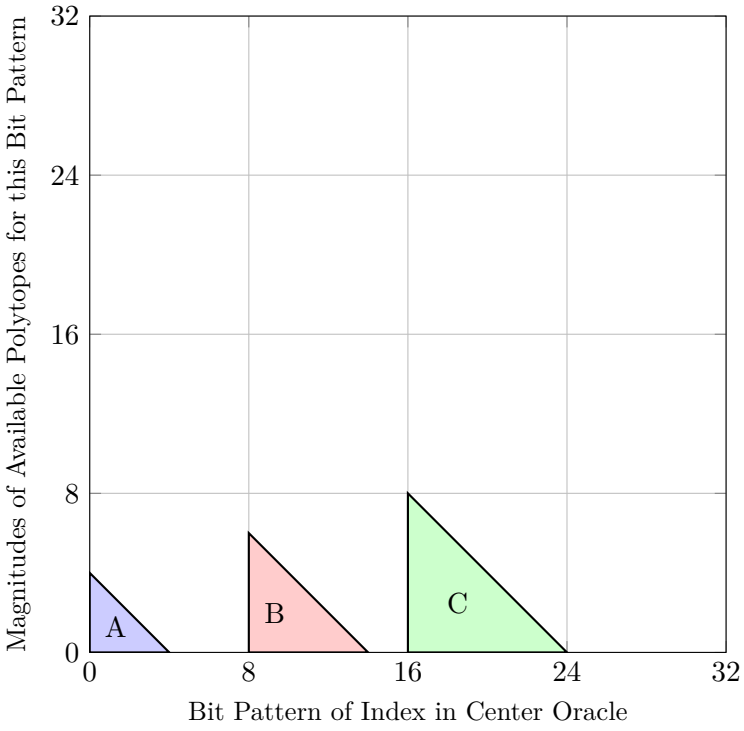



Figure 5: *
Nano mode seed polytopes from indexing Center Oracle.

# 9 Bounded Complexity by Bounding Execution Paths

Traditional memory systems use depth first search through fragmented space, causing performance spikes as utilization grows. Here we ensure every memory request follows a hardcoded path with a fixed execution limit.

## 9.1 Mechanics of Guaranteed Termination

The system bypasses dynamic scanning loops through three distinct structural boundaries:

- **Immediate Rejection at Top Level:** Top-level size admissibility is evaluated using a single comparison against global capacity metrics. If a request exceeds the limit of the Cube data structure, we drops out immediately without executing a deep lookup, establishing a constant-time fast-fail bound.
- **Loopless Grid Sweeps through Row Occupation Counters:** When an allocation request falls within multi-row boundaries, the engine evaluates the row coordinates linearly. Rather than executing an open-ended `while` loop that scans memory maps dynamically, the driver executes through a sequence of unrolled conditional branches. Because these branch paths are laid out explicitly in a fixed sequence, the calculation has a strict, compile-time termination ceiling.
- **Static Matrix Lookups:** Small-scale sector requests route directly to precomputed look-up tables. These tables determine boundary containment and handle alignment splits without using search or sort operations. The system processes the bit pattern of a row through a fixed sequence of matrix indexes, ensuring the path finishes in a predictable number of instructions.

# 10 Desk Check Summary: Nano and Tiny Resolutions ($L \leq \mathcal{A}$)

This section traces two requests through the gates against fully stated occupancy states, so that every step can be reproduced from the stated row words and the Oracle tables of Appendix D. Positions within a row are counted from the left (the most significant bit), as in the tables. A set bit is occupied; a clear bit is free.

### 10.1 The Nano Trace ($L = 5$)

State: the apex counter reads $\Gamma_3 = 1{,}037$. Plane 0 reads $\Gamma_2 = 1{,}017$: its row counters are 27, 30, and then 32 for each of rows 2 through 31. Plane 1 reads $\Gamma_2 = 20$: its row 0 counter is 20 and its remaining rows are empty. Planes 2 through 31 are empty. The two row words that matter are $W_a =$ `0xF1FE7FFF` (plane 0, row 0; 27 occupied, free runs of 3 and 2) and $W_b =$ `0xCFC1F1FC` (plane 1, row 0; 20 occupied, free runs of 2, 5, 3, and 2).

1. **Admissibility (Gate 3):** $5 \leq \mathcal{A}^3 - \Gamma_3 = 32{,}768 - 1{,}037 = 31{,}731$. The descent is permitted.
2. **Plane Steering (Gate 2):** Plane 0 has $1{,}024 - 1{,}017 = 7$ free sectors, at least 5, so it qualifies and is examined first.
3. **Row Steering (Gate 1):** Row 0 of plane 0 has $32 - 27 = 5$ free sectors, so it qualifies. The Center consult on $W_a$ at $L = 5$ returns 255: the row's free sectors stand in runs of 3 and 2, and no run of 5 exists. The suspect is acquitted. Counter support is necessary, not sufficient, exactly as designed.
4. The remaining rows of plane 0 acquit on their counters without a consult: row 1 has 2 free sectors, and rows 2 through 31 have none. Plane 0 is exhausted after 32 row lanes with 1 oracle consult.
5. **Procession Resumes:** Plane 1 qualifies at Gate 2 (1,004 free). Its row 0 qualifies at Gate 1 (12 free). The Center consult on $W_b$ at $L = 5$ returns position 10: conviction. The placement is plane 1, row 0, positions 10 through 14, displacement $1 \times 1{,}024 + 0 \times 32 + 10 = 1{,}034$.
6. **Commitment:** The five bits are set ($W_b$ becomes `0xCFFFF1FC`) and the counters are updated at the single commitment point: row (1, 0) $20 \to 25$, plane 1 $20 \to 25$, apex $1{,}037 \to 1{,}042$.

Accounting: 33 row lanes examined, 2 oracle consults, against the Nano trial ceiling of $\mathcal{A}^2 = 1{,}024$ (Section 3).

### 10.2 The Tiny Trace ($L = 11$)

State: a fresh cube in which the apex counter reads $\Gamma_3 = 11$, all of it in plane 0, row 0, whose word is $W_c =$ `0xE007F800` (11 occupied; free runs of 10 at positions 3 through 12 and 11 at positions 21 through 31). Every other row is empty.

1. **Admissibility (Gate 3):** $11 \leq 32{,}768 - 11$. Permitted.
2. **Steering (Gates 2 and 1):** Plane 0 qualifies (1,013 free); row 0 qualifies (21 free).
3. **First Consult:** The row's first eligible run, at positions 3 through 12, has length 10. It is short of the request by one sector, so no placement is made.
4. **Second Consult:** The trailing run consult on $W_c$ returns $(11, 21)$: a run of 11 ending at the row's last position and beginning at position 21. Conviction. The placement is plane 0, row 0, positions 21 through 31, displacement 21.
5. **Commitment:** The eleven bits are set ($W_c$ becomes `0xE007FFFF`) and the counters are updated: row (0, 0) $11 \to 22$, plane 0 $11 \to 22$, apex $11 \to 22$.

Accounting: 1 row lane examined, 2 consults, against the Tiny trial ceiling of $\mathcal{A}^2 = 1{,}024$ (Section 3).

### 10.3 Verification

Both traces terminate at a conviction, in trial counts far below their class ceilings. The Nano trace also exhibits the designed failure path short of exhaustion: a row whose counter qualifies it and whose geometry acquits it, with the procession resuming intact at the next suspect. The executed step counts depend on the occupancy state, as Section 3 states; the ceilings do not.

# 11 Volumetric Analysis

The following analysis reports the measured behavior of the **Iscandar** architecture under load. The system was exercised across the entire addressable space (32,768 sectors). The measurements are the evidence offered for the bounded-work claims of Section 3: executed step counts vary with the occupancy state, while the ceilings do not.

## 11.1 Methodology

A stress-test audit was performed utilizing a $K = 3$ fractal depth configuration. The system was subjected to sequential allocations, with performance metrics sampled at every $50^{th}$ operation to observe the behavior of the manifold under heavy saturation.

- **Memory Request (Sectors).**
- **Memory Touches (Sectors):** The total register-level interactions required to update the occupancy words and the counter hierarchy ($\Gamma_1$, $\Gamma_2$, $\Gamma_3$).

## 11.2 Graphical Verification

Figure E plots the measured touches per allocation against physical occupancy for the Gaussian Two profile (seed 1; requests of 1 to 1,023 sectors), from the per-allocation records published with the software project. Each point is one successful allocation.

Figure E: Iscandar Performance Profile (Gaussian Two, Measured)

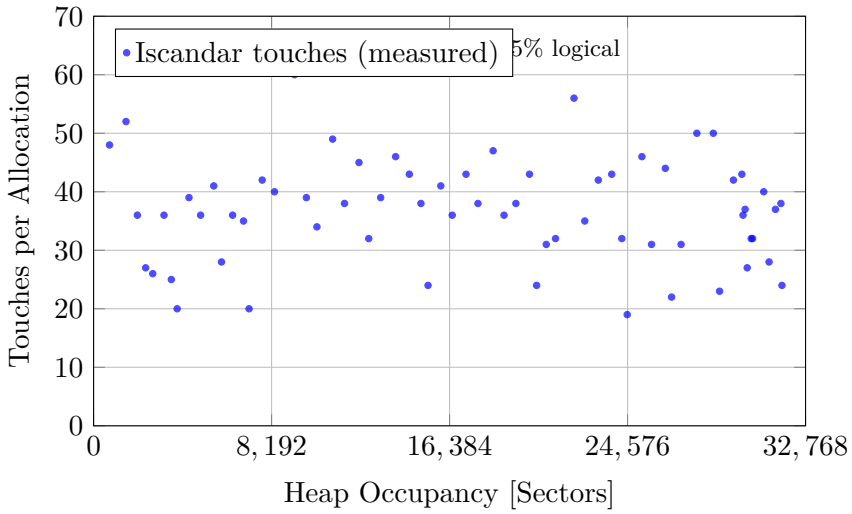



Figure 6: Measured touches per allocation against physical occupancy for the Gaussian Two profile (seed 1; requests of 1 to 1,023 sectors). The run ends where the allocator stalls, at 96.5% physical occupancy, which for Iscandar is also its logical occupancy: the fit is exact.

The executed touches vary with the occupancy state, as Section 3 states; what the profile shows is the band they live in, rising gently with occupancy and remaining far below the trial ceilings.The companion plot for the buddy allocator, on the identical request stream, follows below.

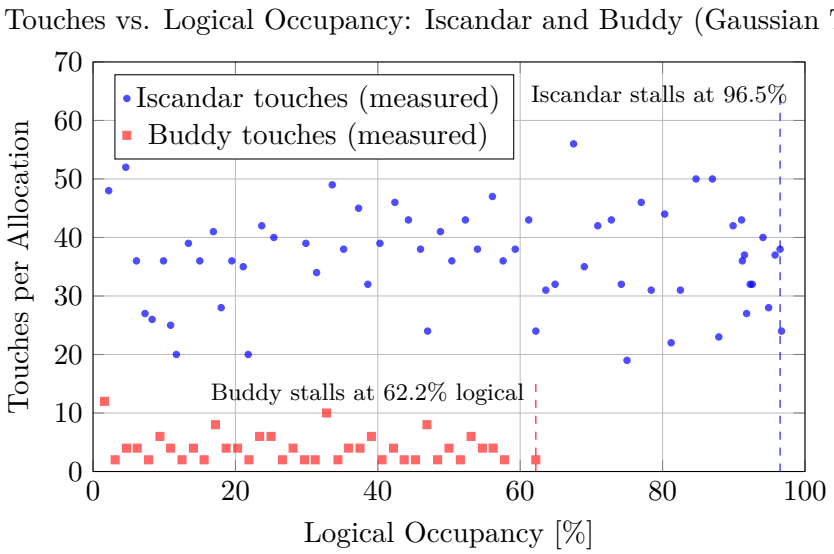



Figure 7: Both allocators on the identical Gaussian Two request stream, touches per allocation against logical occupancy: the share of the pool carrying useful work. Buddy is cheaper per operation, and its trace ends at 62.2% logical (99.2% physical, the difference burned as rounding). Iscandar pays more touches per allocation and carries the stream to 96.5%.

## References


[1] P. R. Wilson, M. S. Johnstone, M. Neely, and D. Boles. *Dynamic Storage Allocation: A Survey and Critical Review*. International Workshop on Memory Management, Springer, 1995. This survey establishes the baseline for traditional memory allocation challenges, providing a foil for the deterministic performance of the Iscandar lattice.

[2] D. E. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley, 3rd Edition, 1997. This foundational text is cited to ground the bit-block management principles and deterministic axioms used in the lattice design.

[3] J. S. Vitter. *External Memory Algorithms and Data Structures*. ACM Computing Surveys, Vol. 33, No. 2, 2001. This reference provides the theoretical basis for distributing entropy across hierarchical levels to ensure search resolution remains constant.

[4] K. C. Knowlton. *A Fast Storage Allocator*. Communications of the ACM, Vol. 8, No. 10, 1965. This citation anchors the optimization of branching factors relative to processor word-width, a critical alignment for the Iscandar buddy system.

[5] M. A. Bender and M. Farach-Colton. *The B-Level: A Cache-Oblivious Data Structure*. Proceedings of the 41st Annual Symposium on Foundations of Computer Science, 2000. This work presents a cache-oblivious hierarchical data structure.

[6] T. J. Anderson. *The Efficiency of Bit-Map Indexing*. Statistical and Scientific Database Management, Vol. 3, 1987. This citation concerns the efficiency of bit-map indexing.

[7] J. M. Robson. *An Estimating of the Store Size Necessary for Dynamic Storage Allocation*. Journal of the ACM, Vol. 18, No. 3, 1971. This reference studies the store size necessary for dynamic storage allocation.

[8] E. D. Jensen, C. D. Locke, and H. Tokuda. *A Time-Driven Scheduling Model for Real-Time Operating Systems*. Proceedings of the IEEE Real-Time Systems Symposium, 1985. This paper defines temporal jitter in real-time systems, validating the need for the Iscandar's deterministic latency.

[9] T. A. Standish. *Data Structure Techniques*. Addison-Wesley, 1980. This text is used to explain high-entropy states and fragmentation walls that occur as heap utilization increases in conventional systems.

[10] I. Puaut. *Real-Time Performance of Dynamic Memory Allocators*. Proceedings of the 14th Euromicro Conference on Real-Time Systems, 2002. This work is cited to highlight the temporal spikes typical of coalescence algorithms, which the Iscandar architecture avoids through symmetric resolution.

[11] D. E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd Edition, 1998. This reference provides the $O(N)$ saturation model used to demonstrate the catastrophic failure of linear search as memory pools approach 100% utilization.

[12] K. C. Knowlton. *Methods for Data Structure Management in Hierarchical Systems*. Bell System Technical Journal, 1965. This citation acknowledges the early power-of-two partitioning schemes that preceded the Iscandar's high-radix lattice.

[13] D. E. Knuth. *Fundamental Algorithms (The Buddy System Formalization)*. Addison-Wesley, 3rd Edition, 1997. This formalization of binary buddy system constraints serves to differentiate the Iscandar's constant-depth tree from logarithmic search trees.

[14] N. Karmarkar. *A New Polynomial-Time Algorithm for Linear Programming*. Combinatorica, Vol. 4, No. 4, 1984. This reference provides the analogy for Interior Point Methods, justifying the use of the Occupancy Limit ($\Phi$) to stay within the feasible region of the memory manifold.

[15] B. S. Everitt. *The Cambridge Dictionary of Statistics*. Cambridge University Press, 1998. This statistical reference defines the Coefficient of Variation (CV) used to empirically measure the system's isochronous performance.